

claude-windows / c1fcb4aa-537d-41fd-858e-53f93349bf5a

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\subagents\workflows\wf_41c8dc8a-8a4\agent-ad0373a5901ba04f1.jsonl`  
- SHA-256: `b5e75907da7939cf61cb766b438eb9550217bb3c25900300c19e12b0ae4870e4`  
- Source modified: `2026-07-02T03:04:11+00:00`  
- Imported at: `2026-07-05T16:48:24+00:00`  
- project: `wf_41c8dc8a-8a4`  
- session_id: `c1fcb4aa-537d-41fd-858e-53f93349bf5a`
```

Transcript

1. user (2026-07-02T03:02:57.784Z)

You are an adversarial verifier in a tech-debt audit. Your default stance: REFUTE. Today is 2026-07-02.
A finder claims the following about the repo at C:/Users/avidu/Projects/khelsutra-guru/sports-data-collector:

```
{  
  "title": "Crawler silently ingests partial matches when a per-set CSV fetch fails (no retry, no flag)",  
  "category": "critical-fix",  
  "severity": "P2",  
  "file": "src/crawlers/badminton_crawler.py",  
  "line": 185,  
  "evidence": "src/crawlers/badminton_crawler.py:184-185: a failed set{N}.csv download is caught ('except Exception as e: print(f\" Warning: Failed to fetch Set {s} CSV: {e}\")') and the loop continues ? the merged CSV then simply lacks that set's rallies. The skip guard at :229 ('if not video_meta or not rallies') only fires when ALL sets failed; a match with e.g. 2 of 3 sets fetched passes validation (:240 validates only the rallies present) and is inserted as if complete (:260-262), with no partial-data marker in the DB and only a scroll-by console line. Downloads use raw urllib.request.urlretrieve with no retry/backoff (:55, :165), so transient GitHub throttling directly manufactures silently-truncated ground truth for the training corpus.",  
  "proposed_fix": "When any per-set fetch fails, either skip the whole match (mirroring the rollback-accounting path at :229-237) or record the video with a non-curated status + a summary_log flag so it is excluded from `curate export`. Retry/backoff convergence onto download_best_video is the parked A1 item; the silent-partial-accept semantics are NOT explicitly covered by A1's scope and are the correctness half worth fixing first (pure-local, testable with a mocked urlretrieve).",  
  "effort": "M",  
  "tracked_in": "docs/NEXT_STEPS.md A1 (retry/backoff only; silent partial-set acceptance not explicitly tracked)"  
}
```

Repo root candidates: C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer, C:/Users/avidu/Projects/khelsutra-guru/khelsutra, C:/Users/avidu/Projects/khelsutra-guru/sports-data-collector, C:/Users/avidu/Projects/khelsutra-guru/rally-annotator, C:/Users/avidu/Projects/khelsutra-guru/sports-obsreport (pick from the finding's repo field; non-git dirs: 'C:/Users/avidu/Projects/khelsutra-guru/Annotation Setup', C:/Users/avidu/Projects/khelsutra-guru/khelsutra-screenshot).
VERIFY against the CURRENT working tree (just pulled to origin master/main): (1) does the cited code actually exist and behave as claimed at file:line? (2) was it already fixed by a recent commit (git log --oneline -15 -- <file>)? (3) is the severity honest - construct the concrete failure scenario or downgrade; (4) is the proposed fix sane and correctly scoped; (5) is it owner-gated per: alpha/serving go-live, Studio P10/P11 corpus-promotion/training, product/strategy/licensing, real cloud spend, counsel ToS/DPA?

HARD RULES: READ-ONLY (no file modifications, no installs, no test-suite runs). gh read-only commands allowed. Your reasoning MUST cite file:line you actually read. CONFIRMED requires you personally saw the defect/debt in current code; if you cannot reproduce the claim from the code, say REFUTED; if a commit fixed it, ALREADY_FIXED.

2. user (2026-07-02T03:03:11.016Z)

Already up to date.

92878ba fix(crawler): guarantee temp-file cleanup in finally (B6) (#38)
d4327c5 chore: address code-review tech debt (config, parsing, deps, fixes)
6dd0d63 feat: always fetch best-resolution video + in-place upgrade path
854b077 Initial commit: sports data collector & curator

3. user (2026-07-02T03:03:11.020Z)

```
1     import os
2     import urllib.request
3     import urllib.parse
4     import csv
5     from typing import Dict, Any, List, Tuple
6     from tabulate import tabulate
7
8     from src.core.config import load_config
9     from src.core.db import SportsDatabase
10    from src.core.schemas.base import VideoMetadata, SportType, LicenseType, CurationStatus
11    from src.core.schemas.badminton import BadmintonRally
12    from src.parsers.badminton.coachai import CoachAIBadmintonParser
13    from src.core.validator import DatasetValidator
14    from src.core.downloader import download_best_video
15
16    class ShuttleSetCrawler:
17        MATCH_CSV_URL = "https://raw.githubusercontent.com/wywyWang/CoachAI-
18        Projects/main/ShuttleSet/set/match.csv"
19        RAW_BASE_URL = "https://raw.githubusercontent.com/wywyWang/CoachAI-
20        Projects/main/ShuttleSet/set"
21
22    def __init__(self, config_path: str = "config/config.yaml"):
23        self.config_path = config_path
24        self.config = self._load_config()
25        self.db_path = self.config.get("database_path", "data/sports_metadata.db")
26        self.db = SportsDatabase(self.db_path)
27        self.validator = DatasetValidator(config_path)
28
29        # Load storage config
30        storage = self.config.get("video_storage", {})
31        self.local_dir = storage.get("local_dir", "./data/videos")
32        self.max_download_size_mb = storage.get("max_download_size_mb", 1000)
33        self.auto_download = storage.get("auto_download", True)
34        # Optional resolution cap (e.g. 1080 to avoid pulling 4K). Unset = best
35        available.
36        self.max_height = storage.get("max_height")
37
38        # Load ingestion limits
39        limits = self.config.get("ingestion_limits", {})
40        self.limit_max_total_size_mb = limits.get("max_total_download_size_mb", 5000)
41        self.limit_max_video_count = limits.get("max_total_video_count", 25)
42
43        os.makedirs(self.local_dir, exist_ok=True)
44        os.makedirs("data/temp", exist_ok=True)
45
46    def _load_config(self) -> Dict[str, Any]:
47        return load_config(self.config_path)
48
49    def fetch_and_curate(self):
50        """Downloads ShuttleSet metadata, crawls match videos, parses rallies, and
```

```

enforces limits."""
48         print("Starting ShuttleSet dataset ingestion...")
49         print(f"Ingestion limits: Max Videos={self.limit_max_video_count}, Max Total
Size={self.limit_max_size_mb_str()}")
50
51         # 1. Download match.csv metadata
52         temp_match_csv = "data/temp/shuttleaset_match.csv"
53         try:
54             print(f"Downloading match catalog from {self.MATCH_CSV_URL}...")
55             urllib.request.urlretrieve(self.MATCH_CSV_URL, temp_match_csv)
56         except Exception as e:
57             print(f"Error fetching match list: {e}")
58             return
59
60         # Read matches
61         matches: List[Dict[str, Any]] = []
62         with open(temp_match_csv, mode='r', encoding='utf-8') as f:
63             reader = csv.DictReader(f)
64             for row in reader:
65                 matches.append(row)
66
67         # 2. Iterate through matches
68         videos_downloaded = 0
69         total_size_downloaded_mb = 0.0
70         total_rallies_indexed = 0
71         db_videos_added = 0
72         db_videos_updated = 0
73
74         # Detailed summary logs
75         summary_log: List[Dict[str, Any]] = []
76
77         print(f"Found {len(matches)} matches in ShuttleSet catalog.")
78
79         for idx, match in enumerate(matches):
80             # Check limits
81             if videos_downloaded >= self.limit_max_video_count:
82                 print(f"\nLimit reached: Maximum video count
({self.limit_max_video_count}) reached. Stopping ingestion.")
83                 break
84             if total_size_downloaded_mb >= self.limit_max_total_size_mb:
85                 print(f"\nLimit reached: Maximum download size
({self.limit_max_total_size_mb} MB) reached. Stopping.")
86                 break
87
88             video_dir_name = match.get("video")
89             youtube_url = match.get("url")
90             match_id = match.get("id")
91
92             # Skip catalog rows missing the fields we depend on rather than
93             # crashing the entire ingestion run.
94             if not video_dir_name or not match_id:
95                 print(f"\n[{idx+1}/{len(matches)}] Skipping catalog row with missing
'video'/'id' column.")
96                 continue
97
98             try:
99                 set_count = int(float(str(match.get("set", "2")).strip() or "2"))
100             except ValueError:
101                 set_count = 2
102             match_name = video_dir_name.replace("_", " ")
103             video_id = f"shuttleaset_{match_id}"
104
105             # ShuttleSet is MIT licensed -> commercially friendly
106             license_type = LicenseType.MIT
107             is_commercial = True

```

```

108
109     print(f"\n[{idx+1}/{len(matches)}] Processing: {match_name}")
110     print(f"  YouTube URL: {youtube_url}")
111
112     # Download video files (if configured)
113     local_path = None
114     download_size_mb = 0.0
115
116     probed_resolution = None
117     probed_fps = None
118     if self.auto_download and youtube_url:
119         # 2.1 Download the best-resolution stream available (MP4). The shared
120         # downloader sorts formats by resolution and records the actual
121         # width/height/fps it obtained ? we never assume a resolution.
122         print("  Downloading best-resolution video...")
123         video_filename = f"{video_id}.mp4"
124         dest_path = os.path.join(self.local_dir, video_filename)
125
126         result = download_best_video(
127             youtube_url, dest_path,
128             max_height=self.max_height,
129             max_size_mb=self.max_download_size_mb,
130         )
131         if result.success:
132             local_path = result.path
133             download_size_mb = result.size_mb
134             probed_resolution = result.resolution_label
135             probed_fps = result.fps
136             videos_downloaded += 1
137             total_size_downloaded_mb += download_size_mb
138             print(f"  Successfully downloaded video "
139                   f"({download_size_mb:.2f} MB, {probed_resolution or 'unknown'
140                   res'})"
141                   f"{'', {probed_fps:g}fps' if probed_fps else ''}")
142         else:
143             print(f"  Failed to download video: {result.error}")
144             local_path = None
145             download_size_mb = 0.0
146
147     # 2.2 Download & Merge stroke CSV files for this match
148     print("  Fetching annotation files...")
149     rallies_merged: List[BadmintonRally] = []
150
151     # Temporary storage to merge set csvs
152     merged_csv_path = f"data/temp/{video_id}_merged.csv"
153     headers = ["set", "rally", "ball_round", "time", "type", "score_a",
154               "score_b"]
155
156     with open(merged_csv_path, 'w', newline='', encoding='utf-8') as mf:
157         writer = csv.writer(mf)
158         writer.writerow(headers)
159
160         for s in range(1, set_count + 1):
161             encoded_dir = urllib.parse.quote(video_dir_name)
162             set_url = f"{self.RAW_BASE_URL}/{encoded_dir}/set{s}.csv"
163             set_temp_path = f"data/temp/{video_id}_set{s}.csv"
164             # B6: cleanup is guaranteed in finally so a row-write or parse
165             # failure mid-set can't orphan the per-set CSV in data/temp.
166             try:
167                 urllib.request.urlretrieve(set_url, set_temp_path)
168
169                 # Read and write rows, adding set number
170                 with open(set_temp_path, 'r', encoding='utf-8') as sf:
171                     sreader = csv.DictReader(sf)
172                     # Normalize fieldnames

```

```

171             sreader.fieldnames = [n.lower().strip() for n in
sreader.fieldnames] if sreader.fieldnames else []
172
173             for row in sreader:
174                 # Write to merged file
175                 writer.writerow([
176                     s,
177                     row.get("rally", "0"),
178                     row.get("ball_round", row.get("ball_seq", "1")),
179                     row.get("time", row.get("timestamp", "0.0")),
180                     row.get("type", ""),
181                     row.get("score_a", row.get("round_score_a", "0")),
182                     row.get("score_b", row.get("round_score_b", "0"))
183                 ])
184             except Exception as e:
185                 print(f" Warning: Failed to fetch Set {s} CSV: {e}")
186             finally:
187                 # Always remove the per-set temp file, even on failure.
188                 if os.path.exists(set_temp_path):
189                     try:
190                         os.remove(set_temp_path)
191                     except OSError:
192                         pass
193
194             # 2.3 Parse merged annotations
195             rallies = []
196             video_meta = None
197             if os.path.exists(merged_csv_path):
198                 parser = CoachAIBadmintonParser(start_buffer=1.0, end_buffer=2.0)
199                 # B6: cleanup is guaranteed in finally so a parse failure can't orphan
200                 # the merged CSV in data/temp.
201                 try:
202                     video_meta, rallies = parser.parse(
203                         merged_csv_path,
204                         video_id=video_id,
205                         video_url=youtube_url,
206                         license=license_type.value,
207                         match_name=match_name,
208                         local_path=local_path
209                     )
210                 except Exception as e:
211                     print(f" Error parsing merged CSV: {e}")
212                     rallies = []
213                 finally:
214                     if os.path.exists(merged_csv_path):
215                         try:
216                             os.remove(merged_csv_path)
217                         except OSError:
218                             pass
219
220             # Override the parser's placeholder resolution/fps with what we actually
221             # downloaded, so the DB reflects the real file (the indexer validates
222             # resolution and keys by it).
223             if video_meta is not None:
224                 if probed_resolution:
225                     video_meta.resolution = probed_resolution
226                 if probed_fps:
227                     video_meta.fps = probed_fps
228
229             if not video_meta or not rallies:
230                 print(" Skipping match due to missing metadata or rallies.")
231                 if local_path and os.path.exists(local_path):
232                     os.remove(local_path)
233                 # Roll back download accounting for the now-discarded file so
234                 # the limit checks and summary stay accurate.

```

```

235         videos_downloaded -= 1
236         total_size_downloaded_mb -= download_size_mb
237         continue
238
239     # 2.4 Validate rallies
240     is_valid, errors = self.validator.validate_badminton_rallies(rallies)
241     if not is_valid:
242         print(f" Warning: Rallies failed validation. Skipping DB insertion.")
243         for err in errors:
244             print(f"     - {err}")
245         if local_path and os.path.exists(local_path):
246             os.remove(local_path)
247             # Roll back download accounting for the now-discarded file.
248             videos_downloaded -= 1
249             total_size_downloaded_mb -= download_size_mb
250         continue
251
252     # 2.5 Insert to Database
253     # Update metrics
254     existing = self.db.get_video(video_id)
255     if existing:
256         db_videos_updated += 1
257     else:
258         db_videos_added += 1
259
260     self.db.insert_video(video_meta)
261     count = self.db.insert_badminton_rallies(rallies)
262     total_rallies_indexed += count
263
264     print(f" Successfully indexed {count} rallies in SQLite database.")
265
266     summary_log.append({
267         "match_name": match_name,
268         "video_id": video_id,
269         "downloaded": local_path is not None,
270         "size_mb": download_size_mb,
271         "rallies": count
272     })
273
274     # 3. Print execution summary
275     self._print_execution_summary(
276         videos_downloaded,
277         total_size_downloaded_mb,
278         total_rallies_indexed,
279         db_videos_added,
280         db_videos_updated,
281         summary_log
282     )
283
284     # Cleanup temp directory (best-effort).
285     try:
286         for f in os.listdir("data/temp"):
287             os.remove(os.path.join("data/temp", f))
288         os.rmdir("data/temp")
289     except OSError:
290         pass
291
292     def limit_max_size_mb_str(self) -> str:
293         return f"{self.limit_max_total_size_mb} MB"
294
295     def _print_execution_summary(self,
296                                videos_downloaded: int,
297                                total_size_mb: float,
298                                total_rallies: int,
299                                added: int,

```

```

300                                     updated: int,
301                                     log: List[Dict[str, Any]]):
302     print("\n" + "="*50)
303     print("                INGESTION EXECUTION SUMMARY                ")
304     print("="*50)
305     print(f"Total Video Files Downloaded: {videos_downloaded}")
306     print(f"Total Download Size           : {total_size_mb:.2f} MB")
307     print(f"Total Rallies Indexed           : {total_rallies}")
308     print(f"Database Matches Added           : {added}")
309     print(f"Database Matches Updated        : {updated}")
310     print("-"*50)
311
312     if log:
313         print("Match Log Breakdown:")
314         table_data = []
315         for item in log:
316             table_data.append([
317                 item["match_name"],
318                 item["video_id"],
319                 "Yes" if item["downloaded"] else "No",
320                 f"{item['size_mb']:.2f} MB",
321                 item["rallies"]
322             ])
323         print(tabulate(table_data, headers=["Match Name", "Video ID", "Downloaded",
324 "File Size", "Rallies"], tablefmt="simple"))
325     else:
326         print("No matches were successfully processed.")
327     print("="*50 + "\n")
328
329     def main():
330         crawler = ShuttleSetCrawler()
331         crawler.fetch_and_curate()
332
333     if __name__ == "__main__":
334         main()

```

4. assistant (2026-07-02T03:03:24.181Z)

The code matches the claim. Let me verify the tracking claim in NEXT_STEPS.md and how curation status / export works, to confirm severity and scoping.

5. user (2026-07-02T03:03:26.144Z)

7-These two structural items were parked during the 2026-06-24 code-health sweep because they are either speculative (no current need) or messy to test thoroughly without the owner's golden corpus. Each has a clear criticality trigger below.

```

8-
9----
10-
11:## A1 ? Crawler retry/backoff convergence
12-
13:**What:** The ShuttleSet crawler (`src/crawlers/badminton_crawler.py`) downloads per-set CSVs
and videos with raw `urllib.request.urlretrieve` ? **no retries, no backoff, no rate-limiting**.
The `fetch` CLI command already has proper retry logic (`_fetch_one_with_retry` in
`src/cli/curate.py`, backed by `download_best_video` in `src/core/downloader.py`). This item
converges the two download paths onto the shared, retried downloader.
14-
15:**Why parked:** The crawler's batch semantics (per-match download limits, cumulative size
accounting, rollback-on-validation-failure) are genuinely different from `fetch`'s single-video
retry. A clean merge needs careful design **and testing against the live ShuttleSet repo**
(which can't be done offline ? it's golden-corpus / live-network territory for the owner). A
half-tested convergence refactor risks breaking the crawl's accounting invariants.
16-
17:**Unpark trigger ? ANY of:**

```

18:- A real crawl run gets throttled and produces partial data (the pain becomes acute).
 19-- The owner has a window to test against the live ShuttleSet repo.
 20-- A second crawler (e.g. for a new dataset) is added, making the duplication cost clearer.
 21-
 22-****When unparked, the design questions to answer first:****
 23:1. Does the shared `download\_best\_video` retry loop subsume the crawler's per-set CSV fetches, or only the video downloads? (CSVs are small; the video path is where throttling bites.)
 24:2. How does retry interact with the cumulative `limit\_max\_total\_size\_mb` / `limit\_max\_video\_count` accounting? (A retried download shouldn't double-count.)
 25-3. Should the crawler gain a configurable inter-match sleep (like `fetch`'s `--sleep`)?
 26-
 27:****Starting point:**** `src/core/downloader.py:download\_best\_video` + `src/cli/curate.py:\_fetch\_one\_with\_retry` are the shared primitives. The crawler's `fetch\_and\_curate` is the integration site.
 28-
 29----
 30-
 31-## A3 ? Read-only DB connection path

6. user (2026-07-02T03:03:27.415Z)

```
sports-data-collector\src\core\db.py:7:from src.core.schemas.base import VideoMetadata,
SportType, CurationStatus, LicenseType
sports-data-collector\src\core\db.py:396:      B5: enum construction
(SportType/LicenseType/CurationStatus) used to raise a
sports-data-collector\src\core\db.py:428:      status = CurationStatus(row['status'])
sports-data-collector\src\core\db.py:458:      def get_videos_by_status(self, status:
CurationStatus, sport: Optional[SportType] = None) -> List[VideoMetadata]:
sports-data-collector\src\core\db.py:622:      def update_status(self, video_id: str, status:
CurationStatus) -> bool:
sports-data-collector\src\crawlers\badminton_crawler.py:10:from src.core.schemas.base import
VideoMetadata, SportType, LicenseType, CurationStatus
sports-data-collector\src\parsers\badminton\coachai.py:8:from src.core.schemas.base import
VideoMetadata, SportType, LicenseType, CurationStatus
sports-data-collector\src\parsers\badminton\coachai.py:188:
status=CurationStatus.STAGING,
sports-data-collector\src\cli\curate.py:15:from src.core.schemas.base import VideoMetadata,
SportType, LicenseType, CurationStatus
sports-data-collector\src\cli\curate.py:43:      instead of stdout + sys.exit. ``status`` is the
CurationStatus the video was recorded with (the
sports-data-collector\src\cli\curate.py:158:      status: CurationStatus =
CurationStatus.CURATED,
sports-data-collector\src\cli\curate.py:166:      ``status`` is the MATURITY GATE: pass
`CurationStatus.STAGING` for a contributed label that
sports-data-collector\src\cli\curate.py:257:      status=CurationStatus.CURATED, # a
manual owner import is already reviewer-vetted
sports-data-collector\src\cli\curate.py:489:      statuses = [CurationStatus.CURATED,
CurationStatus.STAGING]
sports-data-collector\src\cli\curate.py:491:      statuses = [CurationStatus(args.status)]
sports-data-collector\src\cli\curate.py:588:      for status in CurationStatus:
sports-data-collector\src\cli\curate.py:635:      statuses = [CurationStatus.CURATED]
sports-data-collector\src\cli\curate.py:637:      statuses.append(CurationStatus.STAGING)
sports-data-collector\src\cli\curate.py:1131:      staging_videos =
self.db.get_videos_by_status(CurationStatus.STAGING)
sports-data-collector\src\cli\curate.py:1219:
self.db.update_status(video.video_id, CurationStatus.CURATED)
sports-data-collector\src\core\schemas\base.py:24:class CurationStatus(str, Enum):
sports-data-collector\src\core\schemas\base.py:63:      status: CurationStatus =
CurationStatus.STAGING
```

7. user (2026-07-02T03:04:11.231Z)

Structured output provided successfully